

# Engineering Document E23 — Developer Experience & Engineering Standards

**Product Name:** HQ

**Engineering Package:** Version 1.0

---

## Purpose

This document establishes the engineering standards, development workflow, coding conventions, and collaboration practices for HQ.

Its purpose is to ensure that every engineer, regardless of team size or experience, contributes code that is consistent, maintainable, scalable, and production-ready.

Engineering excellence is a core product feature.

---

## Vision

Every engineer joining HQ should be able to:

- Understand the codebase quickly.
- Follow consistent engineering standards.
- Build features confidently.
- Collaborate effectively.
- Release software safely.

Developer experience directly impacts product quality.

---

## Engineering Principles

HQ development follows these principles:

- Simplicity
  - Consistency
  - Reusability
  - Modularity
  - Readability
  - Testability
  - Documentation First
  - Security by Design
- 

## Technology Standards

### Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

### Mobile

- React Native
- Expo
- TypeScript

### Backend

- NestJS
- TypeScript

### Database

- PostgreSQL
- Prisma ORM

### Caching

- Redis

### AI

- AI Gateway

- Gemini
- OpenAI
- Anthropic

#### Infrastructure

- Docker
  - Google Cloud Platform
- 

## Monorepo Structure

```
apps/  
  web/  
  mobile/  
  api/  
  workers/
```

```
packages/  
  ui/  
  sdk/  
  shared/  
  config/  
  prompts/
```

```
docs/  
scripts/  
tools/
```

Every package has a single responsibility.

---

## Package Design Rules

Packages must:

- Be reusable.
- Avoid circular dependencies.
- Export clear public APIs.
- Maintain semantic versioning.
- Include documentation.

Shared code belongs in packages.

---

## TypeScript Standards

Rules:

- Strict mode enabled.
- No implicit any.
- Prefer interfaces for public contracts.
- Use readonly where applicable.
- Avoid unnecessary type assertions.
- Prefer composition over inheritance.

Type safety is mandatory.

---

## Naming Conventions

Files

- kebab-case

Components

- PascalCase

Variables

- camelCase

Constants

- UPPER\_SNAKE\_CASE

Functions

- camelCase

Enums

- PascalCase

## Interfaces

- Prefix with "I" only if required by the team standard; otherwise use descriptive names.

Consistency improves readability.

---

## Folder Organization

Each feature contains:

```
feature/  
components/  
hooks/  
services/  
types/  
utils/  
tests/
```

Related files stay together.

---

## React Standards

Components should:

- Be small.
- Have one responsibility.
- Avoid unnecessary re-renders.
- Prefer hooks.
- Use reusable UI packages.
- Handle loading and error states.

---

## NestJS Standards

Modules contain:

- Controller
- Service
- DTO
- Entity
- Repository
- Tests

Business logic belongs in services.

---

## Prisma Standards

Guidelines:

- One migration per logical change.
- Never edit applied migrations.
- Use descriptive model names.
- Use UUID primary keys.
- Add indexes where appropriate.
- Define relationships explicitly.

Database evolution must remain predictable.

---

## API Standards

REST endpoints follow:

GET

POST

PUT

PATCH

DELETE

Requirements:

- Versioned APIs
  - Input validation
  - Consistent error responses
  - Pagination support
  - Filtering
  - Sorting
- 

## Error Handling

Errors must:

- Be structured.
- Include error codes.
- Avoid exposing sensitive information.
- Be logged.
- Support localization.

Unexpected failures should be recoverable.

---

## Logging Standards

Every log includes:

- Timestamp
- Request ID
- Trace ID
- Organization ID
- User ID (when applicable)
- Severity
- Message

Logs must be structured JSON.

---

## Git Workflow

### Branch Strategy

```
main
↓
develop
↓
feature/*
↓
pull-request
↓
review
↓
merge
```

Direct commits to **main** are prohibited.

---

## Commit Standards

Use Conventional Commits.

Examples:

- feat:
- fix:
- docs:
- refactor:
- test:



- perf:
- chore:
- ci:

Commit history remains meaningful.

---

## Pull Request Requirements

Every Pull Request must include:

- Clear description
- Linked issue
- Screenshots (if UI changes)
- Test results
- Reviewer approval
- Updated documentation (if required)

No self-approval for production merges.

---

## Code Review Standards

Review checklist:

- Readability
- Architecture
- Performance
- Security
- Tests
- Documentation
- Naming consistency
- Reusability

Reviews focus on code quality rather than personal preference.

---

## Documentation Standards

Every module includes:

- Purpose
- Architecture
- Public APIs
- Configuration
- Examples
- Limitations

Documentation evolves with the code.

---

## AI Prompt Standards

Every executive prompt contains:

- Version
- Owner
- Objective
- Input schema
- Output schema
- Safety rules
- Revision history

Prompt changes require review.

---

## Configuration Management

Configuration rules:

- Environment-specific
- Type-safe
- Centralized
- Documented
- Validated during startup

Invalid configuration prevents deployment.

---

## Dependency Management

Dependencies must:

- Be actively maintained.
- Have compatible licenses.
- Pass security scans.
- Avoid duplication.
- Be updated regularly.

Unused dependencies are removed.

---

## Release Management

Semantic Versioning:

Major

Minor

Patch

Every release includes:

- Release notes
  - Migration notes
  - Rollback plan
  - Quality report
- 

## Automation

Automated workflows include:

- Formatting
- Linting
- Testing
- Build validation
- Dependency scanning
- Documentation checks

Automation reduces manual errors.

---

## Engineering Metrics

Track:

- Build success rate
- Deployment frequency
- Code coverage
- Pull request cycle time
- Review turnaround
- Technical debt
- Defect escape rate

Metrics guide engineering improvements.

---

## Onboarding

New engineers receive:

- Development environment
- Architecture documentation
- Coding standards
- Sample tasks
- Mentor support
- Engineering checklist

Fast onboarding improves productivity.

---

## Future Evolution

Supports:

- Internal engineering portal
- Automated code generation
- AI-assisted code review
- Architecture decision records (ADRs)
- Internal package registry
- Developer productivity analytics

Engineering practices evolve alongside the platform.

---

## Success Criteria

The Developer Experience & Engineering Standards succeed when:

- Every contributor follows consistent engineering practices.
  - The codebase remains maintainable as the team grows.
  - Releases are predictable and reliable.
  - Documentation stays current.
  - Developer onboarding is efficient.
  - Engineering quality improves continuously.
- 

## Conclusion

The HQ Developer Experience & Engineering Standards establish a unified engineering culture for the platform.

By combining clear coding conventions, standardized workflows, rigorous code review, documentation requirements, automation, and AI prompt governance, HQ creates a development environment capable of supporting rapid innovation while maintaining enterprise-grade quality, consistency, and long-term maintainability.